Workshop 3

Maria Lucía Castillo García

Índice de contenido

Explicación del flujo....1
Reto: diferencia entre el host EC2 y el contenedor....1
Evidencia de ejecución de cada actividad....3
Parte 1....3
Parte 2....5
Parte 3....7
Parte 4....7
Parte 5....7
Parte 6....8
Parte 7....8
Reto....11

Explicación del flujo

`Internet -> Security Group -> EC2 -> Docker -> Nginx -> Flask`

1. Internet: abro `http://52.23.171.47` en el navegador. Como no pongo puerto, va por el 80.

2. Security Group: AWS revisa si hay una regla que deje pasar ese tráfico. Hay una para el puerto 80 desde cualquier lugar, entonces pasa. Por el 5000 no hay regla, y por eso esa dirección ya no carga.

3. EC2: la petición llega a la instancia por el puerto 80.

4. Docker: ese puerto está conectado con el contenedor de Nginx (el `80:80` del docker-compose).

5. Nginx: recibe la petición y se la pasa a Flask con `proxy_pass http://api:5000`.

6. Flask: responde el JSON y la respuesta hace el mismo camino de vuelta hasta el navegador.

Flask nunca queda expuesto directamente: la única entrada es Nginx.

Reto: diferencia entre el host EC2 y el contenedor

La EC2 es la máquina virtual completa, con su propio Ubuntu, su IP y su nombre (`ip-172-31-30-115`). El contenedor es un proceso aislado que corre dentro de esa EC2, con sus propios archivos y su propio nombre (`07d25c91099c`, que es el ID del contenedor). Por eso los dos hostnames salen distintos.

Como el contenedor está aislado, no puede ver el nombre de la EC2 por sí solo. Para mostrarlo tuve que pasarle el archivo `/etc/hostname` de la EC2 como volumen. También se nota en la hora: el contenedor usa la hora UTC y no la de Colombia.

Evidencia de ejecución de cada actividad

Parte 1

Instalación de Docker con el script oficial

```
ubuntu@ip-172-31-30-115:~$ curl -fsSL https://get.docker.com -o get-docker.sh
ubuntu@ip-172-31-30-115:~$ sudo sh get-docker.sh
# Executing docker install script, commit: ada7a567e59b3935bd601bb21e422da50e80e376
+ sh -c apt-get -qq update >/dev/null
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install ca-certificates curl >/dev/null
Scanning processes...
Scanning linux images...
+ sh -c install -m 0755 -d /etc/apt/keyrings
+ sh -c curl -fsSL "https://download.docker.com/linux/ubuntu/gpg" -o /etc/apt/keyrings/docker.asc
+ sh -c chmod a+r /etc/apt/keyrings/docker.asc
+ sh -c echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu resolute stable" > /etc/apt/sources.list.d/docker.list
+ sh -c apt-get -qq update >/dev/null
+ apt_flags=-y -qq
+ [ -n  ]
+ sh -c DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker-ce docker-ce-cli containerd.io docker-compose-plugin docker-ce-rootless-extras docker-buildx-plugin docker-model-plugin >/dev/null
Scanning processes...
Scanning linux images...
Using systemd to manage Docker service
+ sh -c systemctl enable --now docker.service 2>/dev/null
INFO: Docker daemon enabled and started

+ sh -c docker version
Client: Docker Engine - Community
 Version:           29.9.2
 API version:       1.56
 Go version:        go1.26.10
 Git commit:        a82e528
 Built:             Tue Oct 20 18:45:12 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.9.2
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.10
  Git commit:       0367cbe
  Built:            Tue Oct 20 18:45:12 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.4.2
  GitCommit:        b7a25261e029d68978bbf290df0a62fc819b3c51
 runc:
  Version:          1.5.2
  GitCommit:        v1.5.2-0-g29dd3dc2
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0

================================================================================

To run Docker as a non-privileged user, consider setting up the
Docker daemon in rootless mode for your user:

    dockerd-rootless-setuptool.sh install
```


```
  GitCommit:        v1.5.2-0-g29dd3dc2
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0

================================================================================

To run Docker as a non-privileged user, consider setting up the
Docker daemon in rootless mode for your user:

    dockerd-rootless-setuptool.sh install
```


```
Visit https://docs.docker.com/go/rootless/ to learn about rootless mode.


To run the Docker daemon as a fully privileged service, but granting non-root
users access, refer to https://docs.docker.com/go/daemon-access/

WARNING: Access to the remote API on a privileged Docker daemon is equivalent
         to root access on the host. Refer to the 'Docker daemon attack surface'
         documentation for details: https://docs.docker.com/go/attack-surface/

================================================================================
```

docker --version y hello-world

```
ubuntu@ip-172-31-30-115:~$ docker --version
Docker version 29.9.2, build a82e528
ubuntu@ip-172-31-30-115:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

ubuntu@ip-172-31-30-115:~$ sudo docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

ubuntu@ip-172-31-30-115:~$
```

Usar Docker sin sudo

```
ubuntu@ip-172-31-30-115:~$ sudo usermod -aG docker ubuntu
ubuntu@ip-172-31-30-115:~$ exit
logout
Connection to 52.23.171.47 closed.
```

```
casti@MLC MINGW64 ~/Documents
$ ssh -i curso-ec2-key.pem ubuntu@52.23.171.47
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-1014-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Fri Oct 30 00:34:23 UTC 2026

  System load:  0.33              Temperature:           -273.1 C
  Usage of /:   41.6% of 6.61GB   Processes:             124
  Memory usage: 34%               Users logged in:       0
  Swap usage:   0%                IPv4 address for ens5: 172.31.30.115

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

37 updates can be applied immediately.
9 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Thu Oct 29 22:51:08 2026 from 191.111.250.240
ubuntu@ip-172-31-30-115:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-30-115:~$
```

Parte 2

Creación de los archivos

```
ubuntu@ip-172-31-30-115:~/workshop3/app$ nano app.py
ubuntu@ip-172-31-30-115:~/workshop3/app$ nano requirements.txt
ubuntu@ip-172-31-30-115:~/workshop3/app$ nano Dockerfile
ubuntu@ip-172-31-30-115:~/workshop3/app$
```

app.py

```
 GNU nano 8.7.1
from flask import Flask

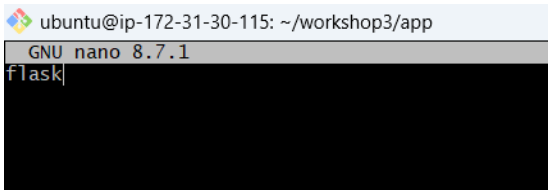
app = Flask(__name__)

@app.get("/")
def home():
    return {"message": "Hola desde EC2"}

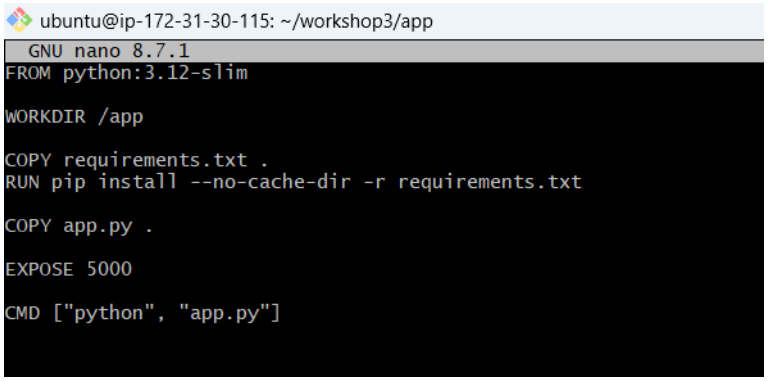
@app.get("/health")
def health():
    return {"status": "ok"}

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

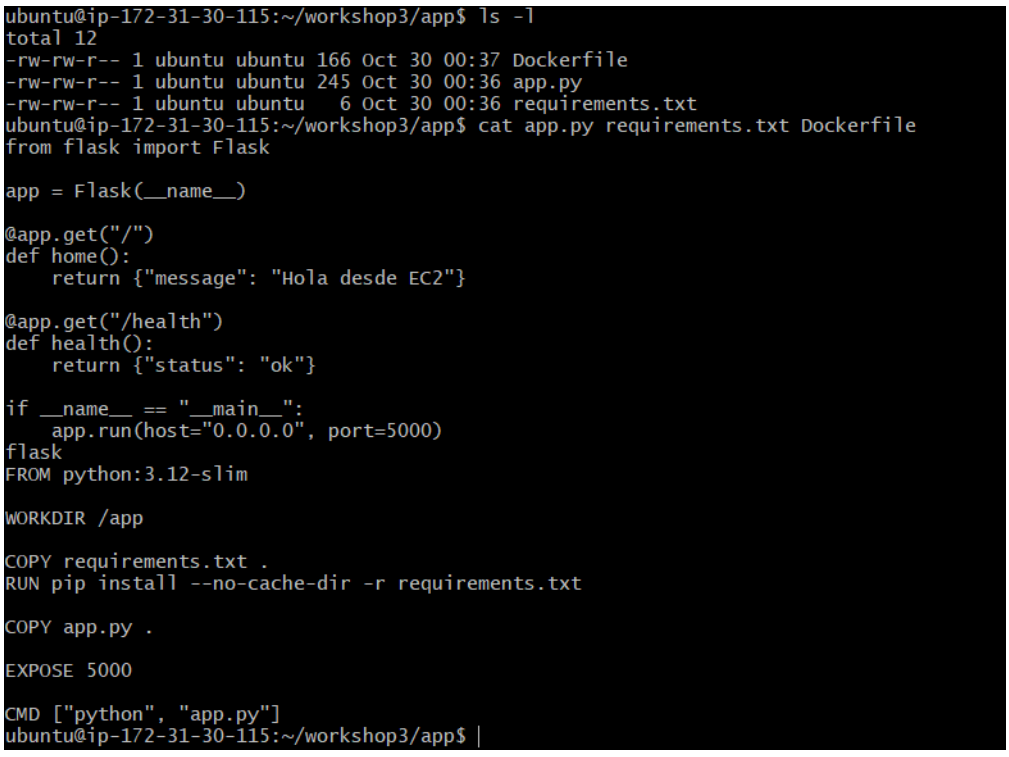
requirements.txt



Dockerfile



Verificación de los archivos



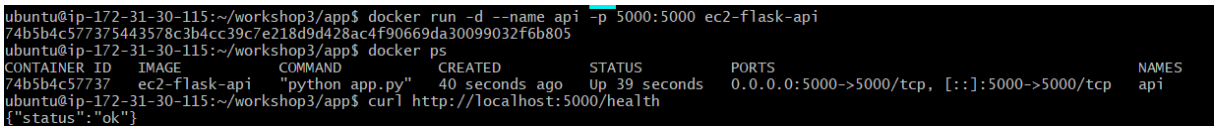
Parte 3

docker build y docker images



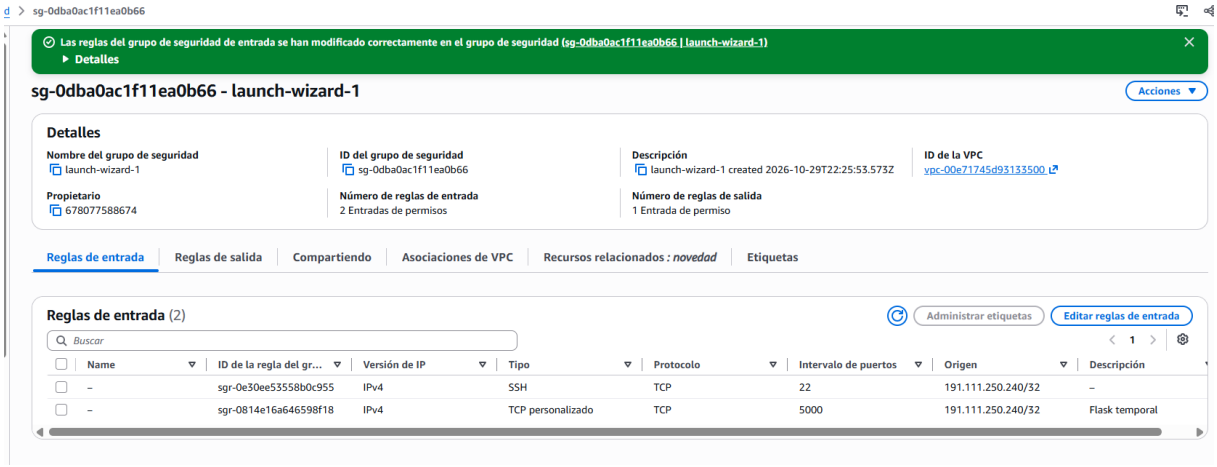
Parte 4

docker run, docker ps y curl

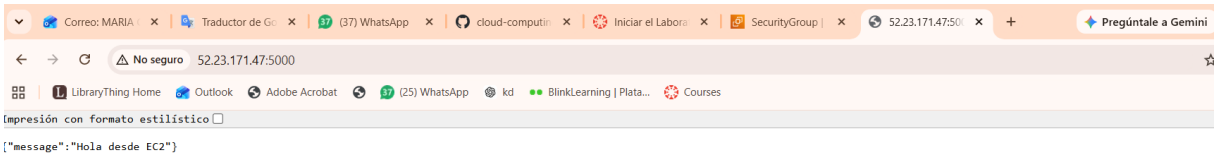


Parte 5

Regla temporal para el puerto 5000



API desde el navegador por el puerto 5000



Parte 6

Borrar el contenedor anterior y crear el docker-compose.yml

```
ubuntu@ip-172-31-30-115:~/workshop3/app$ docker stop api
api
ubuntu@ip-172-31-30-115:~/workshop3/app$ docker rm api
api
ubuntu@ip-172-31-30-115:~/workshop3/app$ nano docker-compose.yml
```

```
ubuntu@ip-172-31-30-115: ~/workshop3/app
  GNU nano 8.7.1
services:
  api:
    build: .
    container_name: api
    ports:
      - "5000:5000"
```

docker compose up y docker compose ps

```
ubuntu@ip-172-31-30-115:~/workshop3/app$ docker compose up -d --build
[+] Building 0.7s (12/12) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 484B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 205B
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:95ca9944be697a0d8ade1197200667fd63208908eb47fb0522fc7a301550c99a
 => => exporting config sha256:04a05698788264eacaa478cfb0742a052a128052ba547e796c68004a63a34fb7
 => => exporting attestation manifest sha256:241ec500518eb5683c29b6bd160c629297a938715e1f803a1ada8a92ec4e8fb4
 => => exporting manifest list sha256:e0bd148e61fa0ea2639d10e2e226707f9c048603ab4ed174095c7b38c0197632
 => => naming to docker.io/library/app-api:latest
 => => unpacking to docker.io/library/app-api:latest
 => resolving provenance for metadata file
[+] up 3/3
 ✔ Image app-api         Built
 ✔ Network app_default   Created
 ✔ Container api         Started
ubuntu@ip-172-31-30-115:~/workshop3/app$
```

```
ubuntu@ip-172-31-30-115:~/workshop3/app$ docker compose ps
NAME      IMAGE     COMMAND           SERVICE   CREATED         STATUS          PORTS
api       app-api   "python app.py"   api       23 seconds ago   Up 23 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
ubuntu@ip-172-31-30-115:~/workshop3/app$
```

Parte 7

Configuración de Nginx

```
ubuntu@ip-172-31-30-115:~/workshop3/app$ cd ~/workshop3
ubuntu@ip-172-31-30-115:~/workshop3$ mkdir nginx
ubuntu@ip-172-31-30-115:~/workshop3$ nano nginx/nginx.conf
ubuntu@ip-172-31-30-115:~/workshop3$ nano docker-compose.yml
```

```
  GNU nano 8.7.1
server {
    listen 80;

    location / {
        proxy_pass http://api:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

docker-compose.yml con Nginx

```
  GNU nano 8.7.1
services:
  api:
    build: ./app
    container_name: api

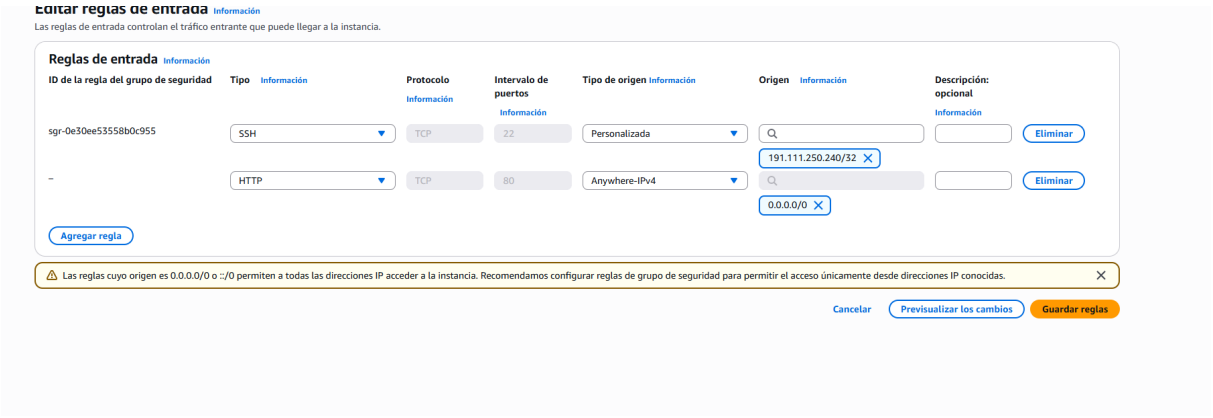
  nginx:
    image: nginx:alpine
    container_name: nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api
```

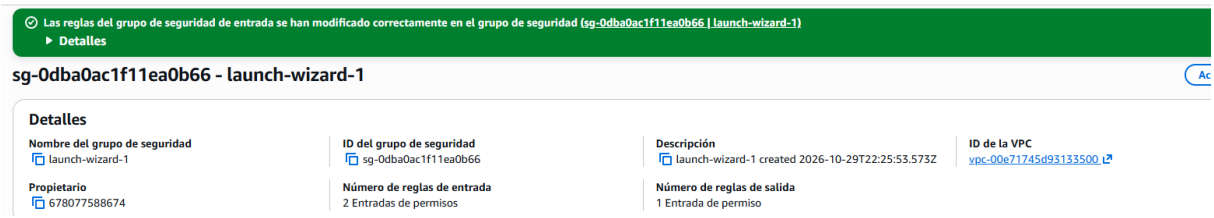
Levantar y probar

```
ubuntu@ip-172-31-30-115:~/workshop3$ docker compose up -d --build
[+] up 11/11
 ✔ Image nginx:alpine Pulled
[+] Building 0.5s (12/12) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 496B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 205B
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => [internal] load build context
 => => transferring context: 63B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:ab8a00c00d67170ce93f3acdf2d635b33e6ffad347094dc1b1b44fde7c1ec084
 => => exporting config sha256:ec1c6db60eb768454a977aadfc8995d78dc2d9156cc5a5f21e113c0e36cdb0e8
 => => exporting attestation manifest sha256:711adc6e319595e3b47ed2fa7158bd9c0fe83cb8e3d7f18b3f69ed80f591cdfc
 => => exporting manifest list sha256:02f735b66ff01b1b05c48f2421e71275d34c64709de71fd83274eb496d5bb54c
 => => naming to docker.io/library/workshop3-api:latest
 => => unpacking to docker.io/library/workshop3-api:latest
 => resolving provenance for metadata file
[+] up 15/15
 ✔ Image nginx:alpine          Pulled
 ✔ Image workshop3-api         Built
 ✔ Network workshop3_default   Created
 ✔ Container api               Started
 ✔ Container nginx             Started
```

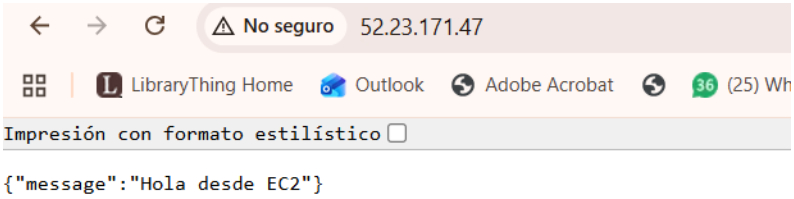
```
ubuntu@ip-172-31-30-115:~/workshop3$ docker compose ps
NAME      IMAGE                                                                                   COMMAND                  SERVICE   CREATED          STATUS          PORTS
api       sha256:02f735b66ff01b1b05c48f2421e71275d34c64709de71fd83274eb496d5bb54c                 "python app.py"          api       49 seconds ago   Up 48 seconds   5000/tcp
nginx     nginx:alpine                                                                            "/docker-entrypoint.…"   nginx     48 seconds ago   Up 48 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp
ubuntu@ip-172-31-30-115:~/workshop3$ curl http://localhost
{"message":"Hola desde EC2"}
ubuntu@ip-172-31-30-115:~/workshop3$ curl http://localhost/health
{"status":"ok"}
ubuntu@ip-172-31-30-115:~/workshop3$
```

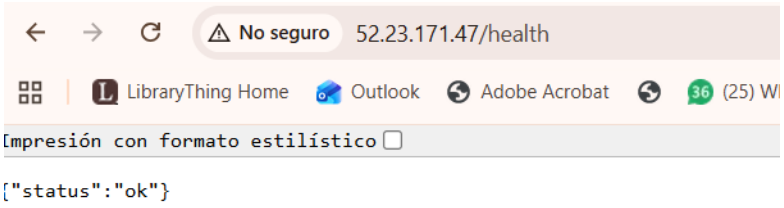
Security group: se abre el 80 y se quita el 5000



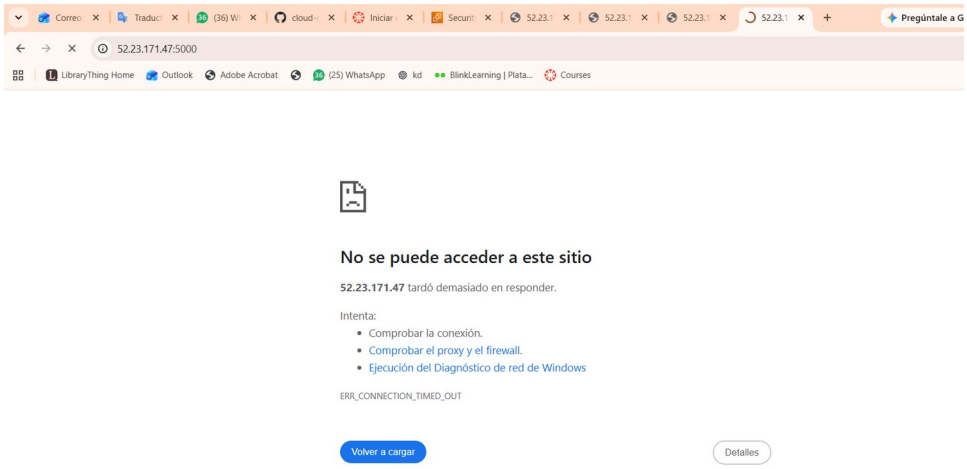


La API por el puerto 80





El puerto 5000 ya no responde



Reto

app.py con los hostnames y la hora

```
ubuntu@ip-172-31-30-115:~/workshop3$ cd ~/workshop3
ubuntu@ip-172-31-30-115:~/workshop3$ > app/app.py
ubuntu@ip-172-31-30-115:~/workshop3$ nano app/app.py
```

```
  GNU nano 8.7.1
import socket
from datetime import datetime

from flask import Flask

app = Flask(__name__)


def hostname_ec2():
    # /etc/hostname de la EC2, montado en el contenedor desde el docker-compose.yml
    with open("/host_hostname") as f:
        return f.read().strip()


@app.get("/")
def home():
    return {
        "message": "Hola desde EC2",
        "hostname_contenedor": socket.gethostname(),
        "hostname_ec2": hostname_ec2(),
        "hora": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    }


@app.get("/health")
def health():
    return {"status": "ok"}


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

docker-compose.yml con el volumen de /etc/hostname

```
ubuntu@ip-172-31-30-115:~/workshop3$ > docker-compose.yml
ubuntu@ip-172-31-30-115:~/workshop3$ nano docker-compose.yml
```

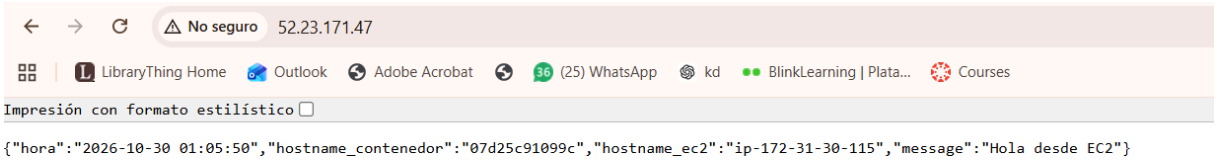
```
 GNU nano 8.7.1
services:
  api:
    build: ./app
    container_name: api
    volumes:
      - /etc/hostname:/host_hostname:ro

  nginx:
    image: nginx:alpine
    container_name: nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api
```

Resultado

```
ubuntu@ip-172-31-30-115:~/workshop3$ docker compose up -d --build
[+] Building 0.7s (12/12) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 496B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 205B
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72aeb34f30d159
 => [internal] load build context
 => => transferring context: 709B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt .
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY app.py .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:b84dc5b5a9174c53216d33ab6501714189f5cb284c1594ad8f029d5b542b98ba
 => => exporting config sha256:fda741b7003f8713765676fb07576ff6abe34e733f116c52dbe1871f3dd91996
 => => exporting attestation manifest sha256:3402ff86b6622f69c51d29da9f77940e6c0e5e6e57b7c109f27e5d7fd7472061
 => => exporting manifest list sha256:b435028c89a0496ea25b5fba5fb3210fd6af3bf160529426b980cc0eccf4bb46
 => => naming to docker.io/library/workshop3-api:latest
 => => unpacking to docker.io/library/workshop3-api:latest
 => resolving provenance for metadata file
[+] up 3/3
 ✔ Image workshop3-api Built
 ✔ Container nginx     Running
 ✔ Container api       Started
ubuntu@ip-172-31-30-115:~/workshop3$
```

```
 => resolving provenance for metadata file
[+] up 3/3
 ✔ Image workshop3-api Built
 ✔ Container nginx     Running
 ✔ Container api       Started
ubuntu@ip-172-31-30-115:~/workshop3$ curl http://localhost
{"hora":"2026-10-30 01:04:24","hostname_contenedor":"07d25c91099c","hostname_ec2":"ip-172-31-30-115","message":"Hola desde EC2"}
ubuntu@ip-172-31-30-115:~/workshop3$
```



{"hora":"2026-10-30 01:05:50","hostname_contenedor":"07d25c91099c","hostname_ec2":"ip-172-31-30-115","message":"Hola desde EC2"}